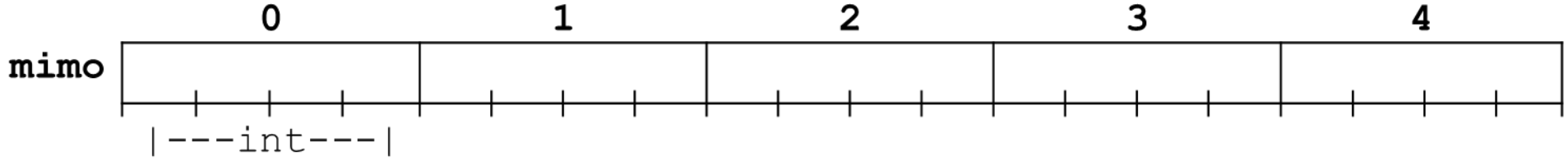
Array

- Introduction to Array
- Declaration and Initialization
- Accessing Elements of an Array
- Operations with an Array
  - Search
  - Insertion and
  - Deletion
- Two Dimensional Array
- String
- Declaration and Initialization
- Access, Input and Output
- String Handling Functions

Array

- An array can hold a series of elements of the same type placed in contiguous memory locations.
- Each of these elements can be individually referenced by using an index to a unique identifier.
- In other words, arrays are a convenient way of grouping a lot of values of same type under a single variable name.
- For example, an array to contain 5 integer values of type int called mimo could be represented like this:



where each blank panel represents an element of the array, that in this case are integer values of type int with size 4 bytes. These elements are numbered/indexed from 0 to 4 since in arrays the first index is always 0, independently of its length.

Array: Declaration

- Like a regular variable, an array must be declared before it is used. A typical declaration for an array is as follows:

```
type name [total_number_of_elements];
```

- where type is a valid data type (like int, float …), name is a valid identifier and the elements field (which is always enclosed in square brackets []), specifies how many of these elements the array can contain.
- Therefore, in order to declare an array called mimo as the one shown in the above diagram it is as simple as:

```
int mimo [5];
```

- The elements field within brackets [] which represents the number of elements the array is going to hold, must be a constant integer value, since arrays are blocks of non-dynamic memory whose size must be determined before execution.

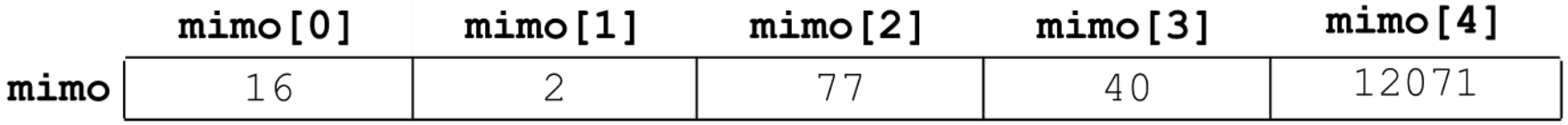
Array: Initialization

When declaring a regular array (int mimo[5];) of local scope (within a function, for example) its elements will not be initialized to any value by default, so their content will be **undetermined** until we store some value in them.

When we declare an array, we have the possibility to assign initial values to each one of its elements by enclosing the values in braces { } separated by comas.

For example: int mimo[5] = { 16, 2, 77, 40, 12071 };

This declaration would have created an array like this:



The number of values between braces { } must not be larger than the number of elements that we declare for the array between square brackets [ ].

Array: Initialization

An array can also be partially initialized. i.e. we assign values to some of the initial elements. For example: int mimo[9] = { 16, 2, 77, 40, 12071 };

This declaration would have created an array like this:

	mimo[0]	mimo[1]	mimo[2]	mimo[3]	mimo[4]	mimo[5]	mimo[6]	mimo[7]	mimo[8]
mimo	16	2	77	40	12071				
						------Uninitialized elements------			

Here the first 5 values are assigned sequentially. The rest 4 elements are unassigned.

Some more initialization –

float x[5] = {5.6, 5.7, 5.8, 5.9, 6.1};

char vowel[6] = {'a', 'e', 'i', 'o', 'u', '\0'};

is equivalent to string declaration: char vowel[6] = "aeiou";

Array: Accessing Element

- The number in the square brackets [ ] of the array is referred to as the 'index' (plural: indices) or 'subscript' of the array and it must be an integer number 0 to one less than the declared number of elements.
- We can access the value of any of its elements individually as if it was a normal variable, thus being able to both read and modify its value. The format is as simple as: name[index]
- For the declaration int mimo[5]; the five (5) elements in mimo is referred in the program by writing: mimo[0] mimo[1] mimo[2] mimo[3] mimo[4]
- Each of the above array elements is an integer and each of them also acts as an integer variable. That is, whatever operation we can perform with an integer variable we can do the same with these array elements using same set of rules.

Array : Accessing Element

- Arrays have a natural partner in programs: **the for loop**. The for loop provides a simple way of counting through the numbers of an index in a controlled way. The for loop can be used to work on an array sequentially at any time during a program.
- It is important to be able to clearly distinguish between the two uses that brackets [ ] have related to arrays.
  - one is to specify the size of arrays when they are declared - char array[5]; Here the index (5) is always an integer constant.
  - the second one is to specify indices for concrete array elements - array[3] = '*'; Here the index (3) is always an integer or an integer expression.
- Consider the following example:

```
char array[5];

array[7] = '*';
```

The assignment of array[7] is clearly wrong as index 7 or element 8 in array[5] does not exists.

Array : Accessing Element (Example)

```
1.  int mimo[5], a, b, i; // declaration of a new array
2.  mimo[2] = 75;         // store 75 in the third element of mimo
3.  a = mimo[2];          // copy/assign a with the third value of mimo
4.  scanf("%d",&mimo[2]); // read a value for the third element of mimo
5.  // read 5 values for the five elements of mimo sequentially
6.  for(i=0; i<5; i++)
7.      scanf("%d",&mimo[i]);
8.  // print 5 values for the five elements of mimo sequentially
9.  for(i=0; i<5; i++)
10.     printf("%d\n",mimo[i]);
11. // some more interesting accesses
12. a = 4;
13. mimo[2] = a;
14. mimo[a] = 3;
15. b = mimo[a-2] + 2; //use of expression in index
16. mimo[mimo[a]] = mimo[2] + b;
```

Operation on Array - Searching

Algorithm:

Input: Array, #elements, item (to search)

Start with the element at index = 0

Step 1: Compare the element at *index* with item. If its equal to item then return *index* with status "**Found**" otherwise go to step 2.

Step 2: Increase index by 1. If index is less than #elements go to step 1 otherwise return -1 with status "**Not found**".

Operation on Array - Insertion

Algorithm:

Input: Array, #elements, item (to insert), position

Step 1 [Shifting]: If *position* is less than *#elements* then
shift each element to right by one position [starting from the last till the element at *position*]

Step 2 [Inserting]: Store *item* at *position*

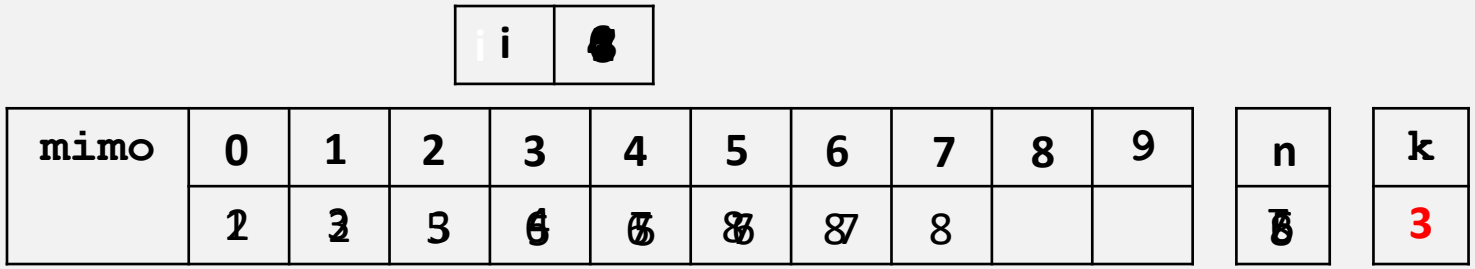
Step 3: Increase *#elements* by 1

Operation on Array - Insertion

```
1.   int k, i, n=5, mimo[10]={2, 3, 5, 6, 7}; //partial initialization; n=total elements
2.   mimo[n++] = 8;          // insert value 8 at the end of the array; increase n;
3.   // insert value 1 at the beginning of array
4.   for(i=n; i>0; i--)       //shift all the values one index forward. i.e. the value
5.      mimo[i] = mimo[i-1]; //in index 1 goes to 2, 2 goes to 3,…, (n-1)th goes to nth.
6.   mimo[0] = 1; n++;        //1 is inserted at index 1; n increases;
7.   // insert value 4 in the middle (index k=3) of the array
8.   k = 3;
9.   for(i=n; i>k; i--)       //shift all the values one index forward. i.e. the value
10.     mimo[i] = mimo[i-1]; //in index k goes to k+1,…, (n-1)th goes to nth.
11.  mimo[k] = 4; n++;        //4 is inserted at index k; n increases;
12.  for(i=0; i<n; i++)       //printing all the values in the array after insert
13.     printf("%d ",mimo[i]);
```

OUTPUT:

1 2 3 4 5 6 7 8



Operation on Array - Deletion

Algorithm:

Input: Array, #elements, position (to delete)

Step 1 [Store the item]: store the value at position to a variable

Step 2 [Shifting]: If *position* is less than *#elements-1* then

shift each element to left by one position [starting from the element at *position+1* till the last element]

Step 3: Decrease *#elements* by 1

Operation on Array - Deletion

```
1.   int k, i, n=8, mimo[10]={1, 2, 3, 4, 5, 6, 7, 8}; //n=total elements.
2.   n--;          // decrease n; last element 8 is no longer part of list.
3.   // delete value 1 from the beginning of array.
4.   n--;          // deleting the value 1 will decrease the total elements n by one.
5.   for(i=0; i<n; i++)    //shift all the values one index backward. The value in index
6.      mimo[i] = mimo[i+1]; //2 goes to 1, 3 goes to 2,…, nth goes to (n-1)th.
7.   k = 2; // delete value 4 from the middle (index k=2) of the array
8.   n--;          // deleting the value 4 will decrease the total elements n by one.
9.   for(i=k; i<n; i++)       //shift all the values one index forward. i.e. the value
10.     mimo[i] = mimo[i-1]; //in index k+1 goes to k,…, nth goes to (n-1)th.
11.  for(i=0; i<n; i++)       //printing all the values in the array after insert
12.     printf("%d ",    mimo[i]);
```

OUTPUT:

2 3 5 6 7

i	6

mimo	0	1	2	3	4	5	6	7	8	9	n	k
	1	2	3	4	5	6	7	8			6	2

Multidimensional Array and Parallel Array

- Multidimensional arrays have more than one dimension
  - All data rows stay together in memory

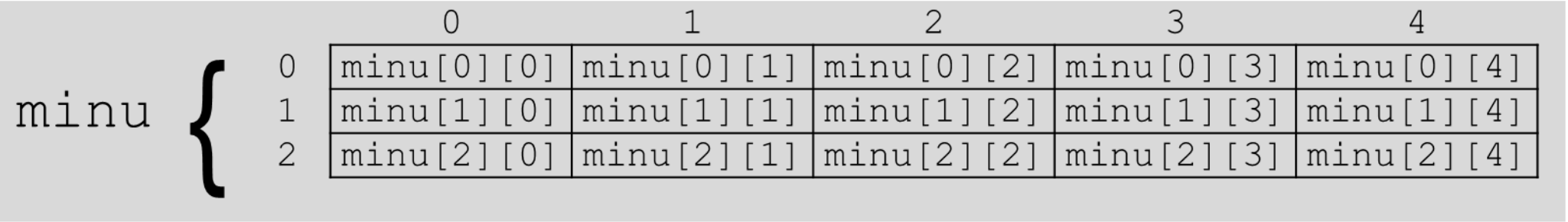
- Parallel arrays are homogeneous data arrays for each field of the record, each having the same number of elements.
  - Data rows are implemented as separate arrays.

Example of Parallel Array

```
int ages[] = {0, 17, 2, 52, 25};
char *names[] = {"None", "Mike", "Billy", "Tom", "Stan"};
int parent[] = {0 /*None*/, 3 /*Tom*/, 1 /*Mike*/, 0 /*None*/, 3 /*Tom*/};
for(i = 1; i <= 4; i++) {
        printf("Name: %s, Age: %d, Parent: %s \n", names[i], ages[i],
        names[parent[i]]);
}
```

Two Dimensional (2D) Array

- Two-dimensional arrays can be described as "arrays of arrays". For example, a 2D array can be imagined as a Two-dimensional table made of elements of same uniform data type.



- minu represents a Two-dimensional array of 3 per 5 elements of type int. The way to declare this array would be: int minu [3][5];
- The way to reference the 2nd element vertically and 4th horizontally or the (2 × 4) 8th element in an expression would be: minu [1][3];
- Generally, for two-dimensional array, 1st dimension is considered as row and the 2nd dimension is considered as column. Here, we have 3 rows and 5 columns.

2D Array: Access Element

Nested loop is used to take input and give output.

The input is taken in row (1st dimension) major. i.e. all the values of row 0 is scanned first, then the values of row 1, and values of row 2. For each row, value at column 0 is scanned first, then the value at column 1, and value at column 2.

In output, array a is used in row major. But array b is used in column (2nd dimension) major. i.e. all the values of column 0 is added first, then the values of column 1, and values of column 2. For each column, value at row 0 is added first, then the value at row 1, and value at row 2.

Consider the following example (the dark rea at the end is the output of this program; the red colored text represents input given by the user):

```
1  // input & output of a 2D array
2  void main (void)
3  {
4    int i, j, a[3][3], b[3][3]={1,3,5,7,9,2,4,6,8};
5    for(i=0;i<3;i++)
6      for(j=0;j<3;j++)
7        scanf("%d", &a[i][j]);
8    for(i=0;i<3;i++)
9      for(j=0;j<3;j++)
10       printf("%d ", a[i][j] + b[j][i]);
11 }
```

```
2 4 6 8 1 3 5 7 9
3 11 10 11 10 9 10 9 17
```

Memory Access

Two-dimensional arrays are just an abstraction for programmers, since we can obtain the same results with a simple array just by putting a factor between its indices: int minu [3][5]; is equivalent to (3 * 5 = 15); int minu [15];

2D Array

```
int minu [3][5], H=3, W=5, n, m, i=0;
void main (void){
  for (n=0; n<H; n++)
    for (m=0; m<W; m++)
      minu [n][m]= ++i;
}
```

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15

1D array

```
int minu [3 * 5], H=3, W=5, n, m, i=0;
void main (void){
  for (n=0; n<H; n++)
    for (m=0; m<W; m++)
      minu [ W * n + m ] = ++i;
}
```

As memory is flat, in both codes the values are actually stored sequentially in the memory (just like the 1D array). The access for the two-dimensional array in that case is just as the indexing of the array,

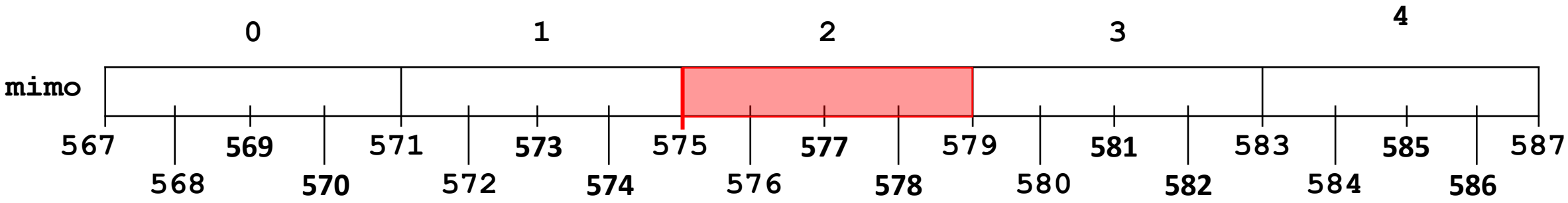
`[(row_index) * (Total_column) + (column_index)]`

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Height	Width	n	m	Width * n + m
3	5	0	0	0

Memory Access

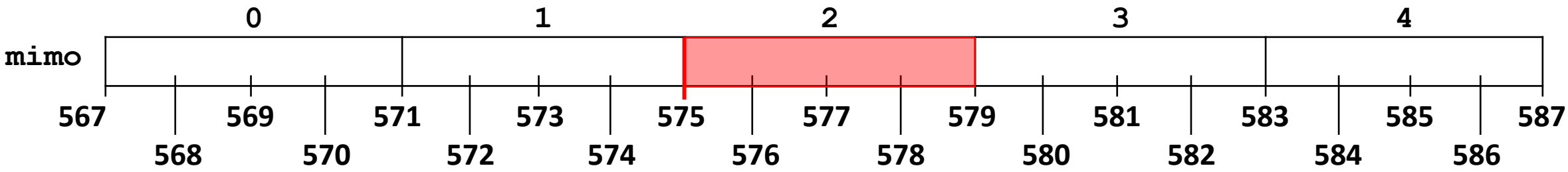
- Memory of each element of an array can be accessed using the **&** operator.
- **&mimo[2]** gives the memory location of the 3rd element of the array **mimo**.
- If the element is more than a byte, it gives the starting byte of the element.
- Let consider the starting address of **int mimo[5]** is **567**.



- **&mimo[2]** will give us the memory location **575**.
- **mimo[2]** will give us 4 bytes (**int**) of information starting from **575** to **579**.
- The name of an array always refer to the starting location of the array. i.e. the first element of the array. So, **mimo = &mimo[0]**.

Memory Access

- &mimo[2] will give us the memory location 575.
- mimo[2] will give us 4 bytes (int) of information starting from 575 to 578.
- The name of an array always refer to the starting location of the array. i.e. the first element of the array. So, mimo = &mimo[0].



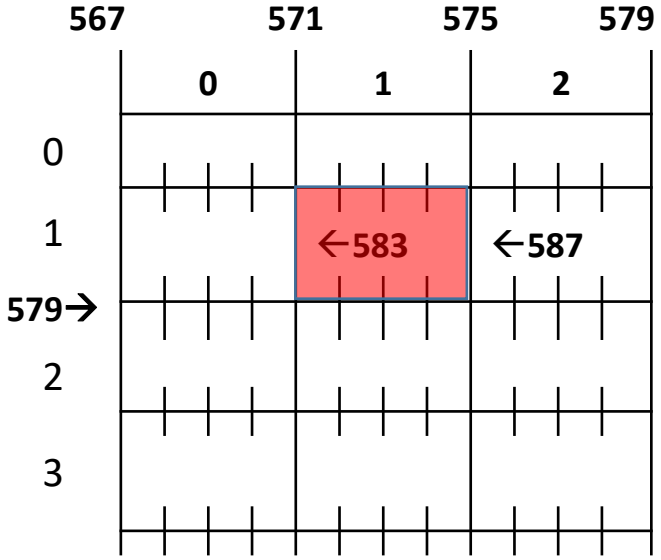
&array[index]=start_location_array + index * size_of_data

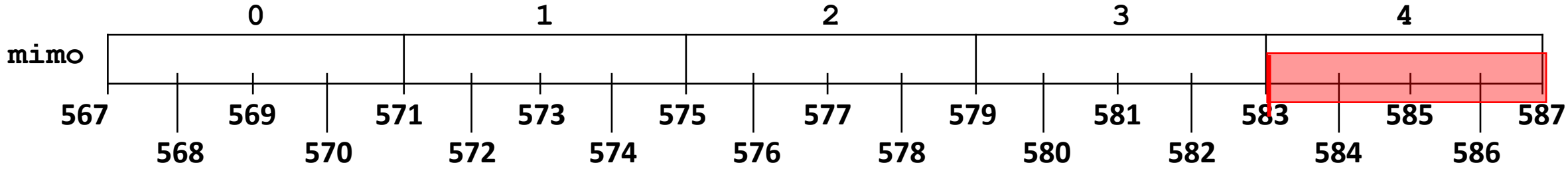
⇒&mimo[2] = mimo (or &mimo[0]) + 2 * sizeof(int)

⇒&mimo[2] = 567 + 2 * 4 = 575

Memory Access

- Consider a 2D array mimo[R][C] each element addressed by &mimo[i][j], where R=total element in 1st dimension, C=total element in 2nd dimension, 0≤ i < R, 0 ≤ j < C.
- Let int mimo[4][3]; Here, mimo or &mimo[0][0] gives us the starting memory location 567.
- mimo[1][1] will give us 4 bytes (int) of information starting from 583 to 587.





&array[i][j]=start_location + (i * (C * size_of_data)) + (j * size_of_data)

⇒&mimo[1][1] = mimo + (1 * (3 * sizeof(int))) + (1 * sizeof(int))

⇒&mimo[1][1] = 567 + (1 * 3 * 4) + (1 * 4) = 583

Memory Access

- There is a general way to access the memory location of a 2 dimensional array.
- For an array int mimo[R][C]; and $0 \leq i < R$; $0 \leq j < C$.
  - mimo[i] = &mimo[0][j] represents the starting address of ith row.
  - mimo[i] skips **i** number of rows each with **C** number of elements from the **start_location** of the array.
  - So, mimo[i] = start_location + (i * C elements), where C elements are counted in bytes based on the **size_of_data**, here int.
  - So, mimo[i] = start_location + (i * (C * size_of_data)).
  - So, mimo[i] = mimo(or &mimo[0][0]) + (i * (C *sizeof(int))).
- A 2D array is also referred as an array of arrays. i.e. an array of which each element is another array.